macOS OpenXR game example

Triscuitcircuit

October 21, 2026

Contents

1 Requirements **2**

2 Run **2**

3 How Newt fits **2**

4 Bundle an exported macOS game **3**

5 Validation status **4**

6 References **4**

7 Editor Play integration **4**

A small target game: aim a tracked controller at the sphere and squeeze the trigger to score. The camera and targets use world coordinates; the game does not move the world with head rotation. There is no artificial locomotion.

1 Requirements

- Godot 4 with the OpenXR module and Vulkan renderer available on macOS. An engine built without XR will not work; if building from source, enable `module_openxr_enabled=yes` and `disable_xr=no`.
- A compatible, separately supplied macOS OpenXR runtime providing Vulkan graphics, tracked headset/controllers, and a working USB headset transport.
- Matching architectures for the engine, runtime, Vulkan implementation, and all dynamic libraries (for example all arm64).
- A connected and awake headset. Close any dashboard/application holding an exclusive runtime session before starting the game.

This example uses the engine's default OpenXR action map, including `trigger`, `left_hand`, `right_hand`, and aim poses. If you customize it, retain these actions/bindings or update the script.

2 Run

```
python3 launch.py --runtime /absolute/path/to/runtime.json --godot /path/to/Godot.app/C
```

Add `--icd /absolute/path/to/MoltenVK_icd.json` if your runtime deployment requires it. For an exported macOS game, use `--game /path/to/Game.app/Contents/MacOS/` instead of `--godot`.

The launcher uses argument arrays, validates the manifest, and sets the runtime before starting the engine. OpenXR and XR shaders are enabled in project settings. Use Mobile + Vulkan for this runtime path rather than the native Metal renderer.

3 How Newt fits

Godot owns its OpenXR session and render loop. **Do not call `Newt_Run()` inside this Godot game:** Newt's current dashboard also owns a session

and rendering loop. Use the dashboard as a separate launcher and hand off the headset to Godot. This example uses the external runtime directly; it does not embed Newt's dashboard.

`Newt::usb` is available to native host tools for generic USB operations. Do not claim the headset's USB interface from a game while its runtime owns it. libusb transport alone does not implement Quest pairing, tracked poses, encoded video or timewarp.

Embedding Newt panels directly in Godot would need a new GDExtension that exposes capture frames and panel state as Godot textures/nodes, uses Godot's existing OpenXR session, and avoids starting Newt's own Vulkan renderer. That adapter is not implemented here.

4 Bundle an exported macOS game

A deployment layout can be:

```
Game.app/Contents/
  MacOS/Game                          # exported Godot executable
  Resources/Game.pck                  # as generated by the export
  Resources/OpenXR/runtime.json
  Resources/OpenXR/MoltenVK_icd.json  # only if using an external ICD
  Frameworks/XR/                      # external runtime and its dependency dylibs
  Resources/Licenses/                 # notices for every shipped component
```

1. Export the project using matching macOS export templates. Preserve the export's actual executable and resource filenames.

2. Supply your runtime as a separately built artifact with its dependency list, license notices, source-distribution materials where required, and supported headset/architecture documented. This repository does not contain that artifact.

3. Copy its dylibs into `Frameworks/XR`. Rewrite non-system load paths to bundle-relative `@loader_path` / `@rpath` paths. The runtime JSON's `runtime.library_path` can point to `../../Frameworks/XR/libopenxr_monado.dylib` if that is the selected runtime's real filename. Rewrite an external ICD manifest to its actual bundled library path too.

4. Use a small native launcher as the app's `CFBundleExecutable` to resolve the bundle path, set `XR_RUNTIME_JSON` (and the ICD selection when needed), then exec the exported game executable. `launch.py`

demonstrates the environment/argument contract for development; do not depend on users having Python installed for a shipping app. Finder launches do not inherit variables from your terminal.

5. Sign nested libraries and executables, then the outer app; notarize your release as appropriate. Verify all dependencies with `otool -L`, and test on a clean Mac without Homebrew or developer environment variables.

Do not assume changing a build flag removes GPL code: audit the actual runtime artifact. Newt's MIT license does not relicense runtime code or libusb.

5 Validation status

The project can be imported and run without XR for script/scene checks. A successful desktop check does not validate Vulkan/OpenXR initialization, controller bindings, frame timing, USB transport, or headset audio. Those need a hardware run with the selected engine/runtime combination.

6 References

- Godot XR setup
- Godot OpenXR build configuration

7 Editor Play integration

Use the Newt Play Bridge plugin to discover/start Newt automatically on Play, hand off the headset, and return after the game exits. The session manager also creates development app bundles that reuse the installed runtime.